Ejercicio Práctico Clase 3

GRUPO 3

Integrantes:

- Juan Carlos Heredia Vallejo
- Jonathan Fernando Tisalema Lascano

Objetivo: Consolidar los conceptos de limpieza y transformación de datos, prevención de data leakage y diseño de pipelines reproducibles, integrando herramientas programáticas como Python y entornos visuales como Orange Data Mining. Para el desarrollo del trabajo, se recomienda utilizar Google Colab, que permite escribir y ejecutar código Python en la nube desde el navegador web, así como integrar bloques de texto en formato Markdown para la documentación técnica y metodológica.

Fase I: Análisis en Python (Pipeline Profesional):

1. Seleccione un dataset estructurado de clasificación (mínimo 10 variables).

   Se ha seleccionado el conjunto de datos Loan Default, el cual está orientado al análisis de riesgo crediticio cuyo objetivo es predecir si una persona incumplirá el pago de un préstamo (variable objetivo: default, donde 1 [incumple] y 0 [no incumple]) tomando en cuenta cierta información específica de cada persona. El dataset contiene 255.347 registros y 18 variables, que incluyen características demográficas y financieras del solicitante (edad, ingresos, puntaje de crédito, meses empleado, estado civil, nivel educativo), variables relacionadas con el prés
   tamo (monto, tasa de interés, plazo, propósito del préstamo) y condiciones financieras adicionales (relación deuda-ingreso, número de líneas de crédito, existencia de hipoteca o codeudor). Estas variables permiten analizar los factores que influyen en el incumplimiento y construir modelos predictivos para la gestión eficiente del riesgo crediticio.

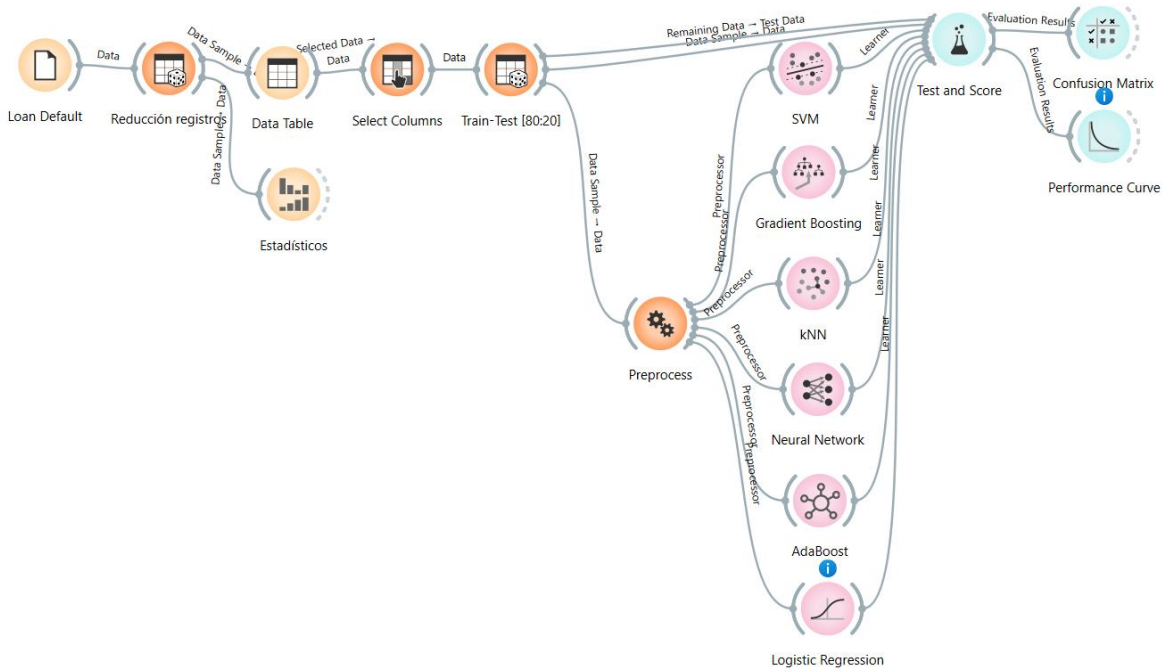
2. Realice análisis exploratorio básico: distribución, valores faltantes, outliers.
3. Implemente dos enfoques:
   a. Enfoque incorrecto (con posible data leakage).
   b. Enfoque correcto utilizando Pipeline de sklearn.
4. Compare métricas utilizando validación cruzada.
5. Analice los resultados y explique el impacto arquitectónico.

El desarrollo de esta fase puede verse en el notebook adjunto, o véase en línea [ver archivo].

Fase II: Diseño Visual en Orange

1. Construya un flujo con los siguientes widgets:
    a. File
    b. Select Columns
    c. Normalize
    d. Test & Score
    e. Confusion Matrix

Se ha realizado el siguiente flujo:



'Loan Default' corresponde al **literal 'File'**, donde se cargan los datos y se define a la variable Default como variable objetivo (target):

Info
255347 instances
17 features (no missing values)
Data has no target variable.
1 meta attribute

Columns (Double click to edit)

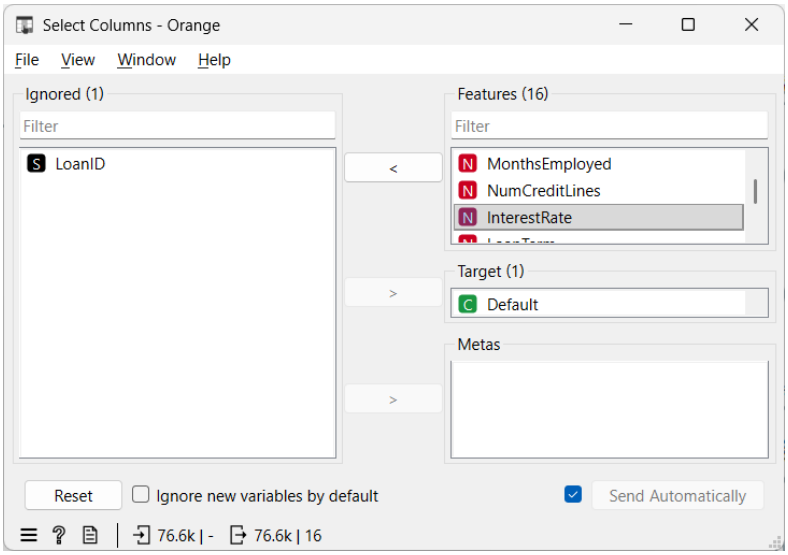
	Name	Type	Role	Values
12	MaritalStatus	categorical	feature	Divorced, Married, Single
13	HasMortgage	categorical	feature	No, Yes
14	HasDependents	categorical	feature	No, Yes
15	LoanPurpose	categorical	feature	Auto, Business, Education, Home, Other
16	HasCoSigner	categorical	feature	No, Yes
17	**Default**	**categorical**	**target**	**0, 1**
18	LoanID	text	meta	

Debido a que el tamaño de la data es grande y el recurso de memoria se presentaba demandante, se procede a realizar una muestra de la data [en el ítem Reducción registros], a fin de reducir los datos a trabajar (considerando el 30% de los datos originales [76.6k registros]) y agilizar el ejercicio práctico.

Posterior se presentan los ítems [**Data Table**] para visualizar la data filtrada y [**Estadísticos**] para revisar si existen datos nulos, en este caso todas las variables tienen datos completos. **Nota:** La variable objetivo es presenta un desbalance en la presencia de sus categorías.

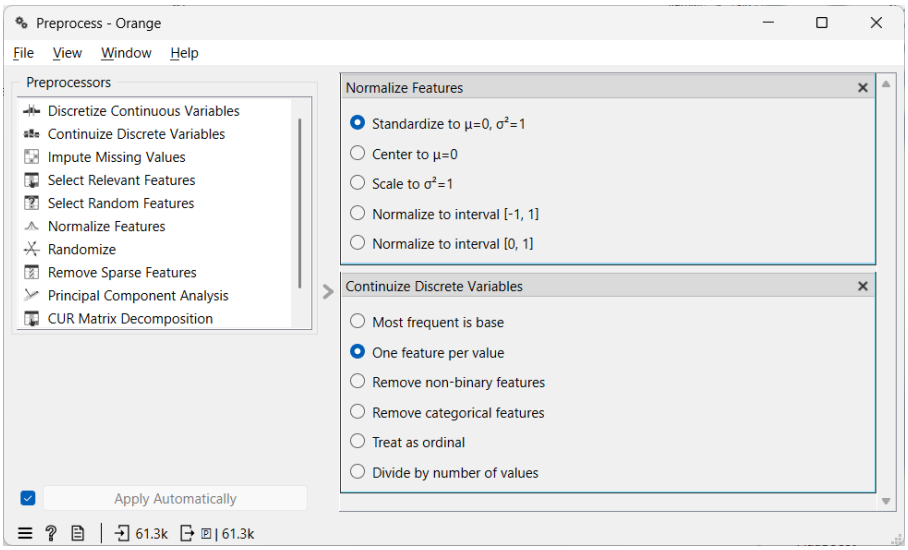
Column	Distribution	Mean	Mode	Median	Dispersion	Min.	Max.	Missing
Default			0		0.359			0 (0 %)

Cumpliendo con el **literal 'Select Columns'**, se agrega el ítem [Select Columns] y se excluye la variable id que no presenta una información valiosa para la estructura del modelo. No se han excluido más variables ya que se consideran información importante para catalogar a una persona como buen o mal pagador.

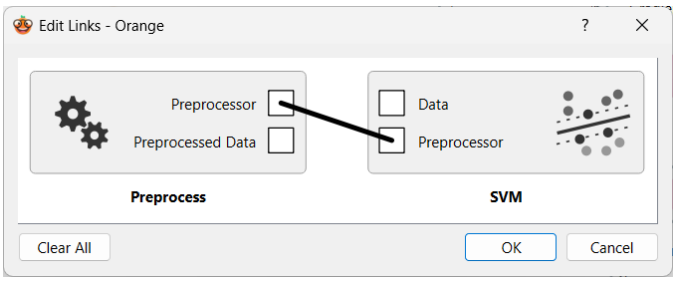


Posterior se encuentra el ítem [Train-Test] en el que se divide la data, en conjuntos de entrenamiento y prueba, en una razón de 80:20 respectivamente.

La data de entrenamiento tendrá un preprocesamiento, en el ítem [**Preprocess**], en el que se realiza un procedimiento de **Normalize** a fin de estandarizar las escalas de las variables numéricas, y en conjunto, un procedimiento **Continuize** a fin de crear variables dummies/binarias para las variables que presenten más de 2 categorías:



Una vez listos los valores, se establecen algunos **modelos de predicción** (SVM, Gradient Boosting, kNN, Neural Network, AdaBoost y Logistic Regression), para ello la relación entre Preprocess y los modelos será:



Debido a que utilizar 'Preprocessor' toma los datos originales y ejecuta Normaliza y Continuize internamente, esto evita la fuga de datos y evidencia la sensibilidad de normalizar/estandarizar variables numéricas (se comparará posteriormente al incluir y no incluir el ítem [Preprocess].

Finalmente se implementan los ítems [Test & Score] el cual nos brinda estadísticos de los modelos utilizados, [Confusion Matrix] que nos ayuda interpretar de manera visual el rendimiento del modelo y [Performance Curve] que brinda la métrica AUC la cual indica el rendimiento de clasificación del modelo, los resultados se presentan en el siguiente punto.

2. Compare al menos tres modelos.

En el ítem [Test & Score] se presenta los siguientes resultados de los modelos empleados:

Model	AUC	CA	F1	Prec	Recall	MCC
Gradient Boosting	0.762	0.885	0.843	0.856	0.885	0.177
Logistic Regression	0.759	0.884	0.837	0.855	0.884	0.138
Neural Network	0.721	0.875	0.848	0.839	0.875	0.193
SVM	0.625	0.760	0.785	0.818	0.760	0.117
kNN	0.596	0.860	0.834	0.818	0.860	0.110
AdaBoost	0.557	0.802	0.808	0.815	0.802	0.108

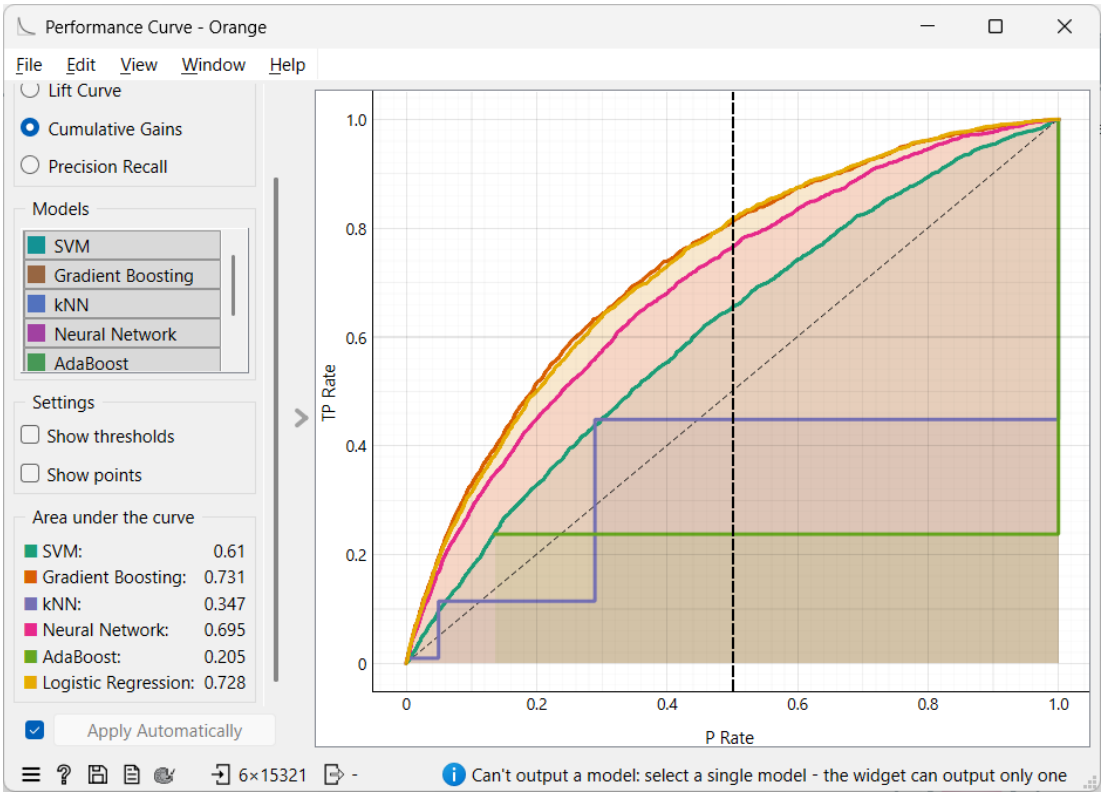
En la tabla se presentan distintos estadísticos de evaluación para comparar modelos de clasificación. El AUC (Area Under the Curve) mide la capacidad del modelo para distinguir entre clases; valores más cercanos a 1 indican mejor desempeño discriminativo. La CA (Classification Accuracy) representa la proporción total de predicciones correctas. El F1-score es la media armónica entre precisión y recall, útil cuando existe desbalance de clases. La Precisión (Precision) mide qué proporción de las predicciones positivas es correcta, mientras que el Recall (Sensibilidad) indica qué porcentaje de los casos positivos reales fue correctamente identificado. Finalmente, el MCC (Matthews Correlation

Coefficient) es una métrica más robusta que considera verdaderos y falsos positivos y negativos, siendo especialmente útil en conjuntos de datos desbalanceados; valores cercanos a 1 indican mejor desempeño global.

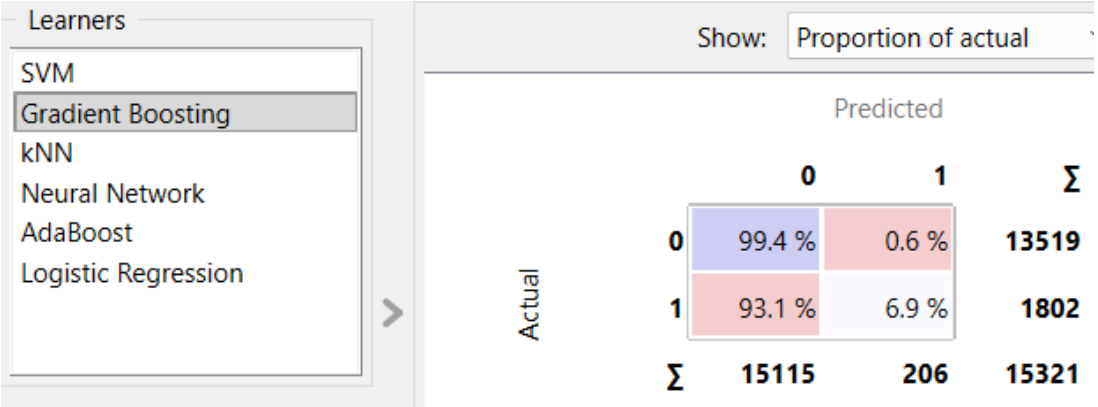
Dado que la variable objetivo es desbalanceada, es apropiado priorizar métricas robustas como AUC, F1 y especialmente MCC. Bajo este enfoque, los tres mejores modelos son:

- Gradient Boosting mantiene el mejor AUC (0.762) y la mayor exactitud (0.885), lo que confirma su alta capacidad para discriminar entre clientes que incumplen y no incumplen. Su F1 (0.843) y MCC (0.177) reflejan un desempeño equilibrado en un contexto de clases desbalanceadas.
- Neural Network vuelve a destacar en métricas de balance, obteniendo el mayor F1 (0.848) y el mejor MCC (0.193), lo que indica que maneja relativamente mejor el compromiso entre falsos positivos y falsos negativos.
- Logistic Regression presenta resultados muy cercanos a Gradient Boosting (AUC 0.759, CA 0.884), mostrando estabilidad y buena capacidad predictiva con la ventaja de ser un modelo interpretable.

En contraste el rendimiento de predecir adecuadamente a personas que puedan presentar un incumplimiento, se visualiza la curva ROC (mejor entre más alejada se encuentre de la diagonal principal):



Los mejores modelos y muy cercanos son Gradient Boosting y Logistic Regression. Estos resultados pueden interpretarse visualmente mediante la matriz de confusión:

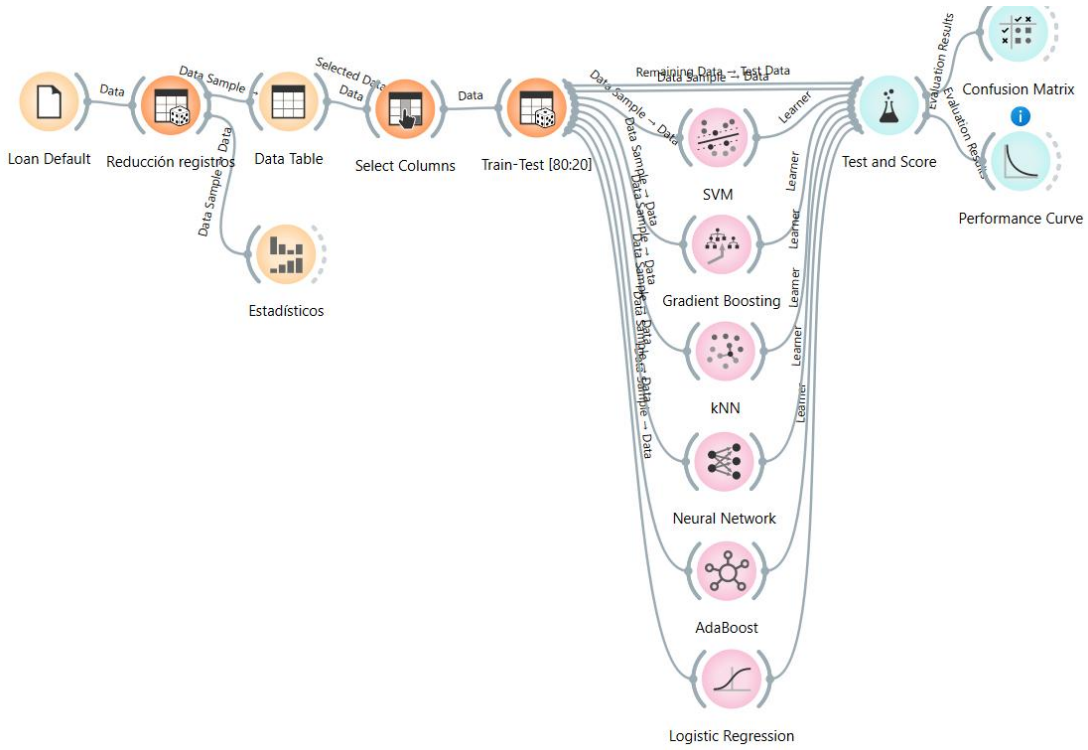


Si bien el estadístico AUC es mayor a 0.7, suponiendo un buen rendimiento, en la matriz de confusión podemos observar que solo un 7% de los datos reales de incumplimiento son catalogados correctamente.

Los modelos podrían mejorar al implementar algoritmos que balanceen la data o considerar variables adicionales que expliquen mejor la varianza del modelo.

3. Analice diferencias con y sin normalización.

A fin de identificar diferencias por fuga de datos o falta de preprocesamiento de datos, se realiza el mismo ejercicio excluyendo el ítem [Preprocess], de tal manera que el flujo a utilizar es:


Loan Default
Data
Reducción registros
Data Sample
Data Table
Selected Data
Data
Select Columns
Data
Train-Test [80:20]
Data Sample → Data
Remaining Data → Test Data
Data Sample → Data
Data Sample → Data
Data Sample →

4. Discuta qué modelos son sensibles al escalamiento y por qué.

Model	AUC	CA	F1	Prec	Recall	MCC
Gradient Boosting	0.762	0.885	0.843	0.856	0.885	0.177
Logistic Regression	0.758	0.884	0.837	0.855	0.884	0.138
Neural Network	0.721	0.875	0.848	0.839	0.875	0.193
SVM	0.625	0.760	0.785	0.818	0.760	0.117
AdaBoost	0.559	0.803	0.809	0.815	0.803	0.110
kNN	0.532	0.857	0.827	0.806	0.857	0.057

Sin normalizar

Model	AUC	CA	F1	Prec	Recall	MCC
Gradient Boosting	0.762	0.885	0.843	0.856	0.885	0.177
Logistic Regression	0.759	0.884	0.837	0.855	0.884	0.138
Neural Network	0.721	0.875	0.848	0.839	0.875	0.193
SVM	0.625	0.760	0.785	0.818	0.760	0.117
kNN	0.596	0.860	0.834	0.818	0.860	0.110
AdaBoost	0.557	0.802	0.808	0.815	0.802	0.108

Normalizado

Los modelos sensibles a la normalización son aquello que su mecanismo de aprendizaje depende directamente de la magnitud y escala de las variables de entrada.

kNN: el algoritmo clasifica nuevas observaciones en función de distancias entre puntos. Si una variable tiene valores mucho más grandes que otra, dominará el cálculo de distancia y tendrá un peso mayor en el modelo, al normalizar, todas las variables quedan en una escala comparable, permitiendo que cada característica contribuya de manera equilibrada y mejorando el rendimiento.

Redes Neuronales: son sensibles debido al proceso de entrenamiento mediante descenso de gradiente. Cuando las variables presentan escalas muy distintas, los pesos asociados a variables grandes pueden ajustarse más rápidamente que otros, generando oscilaciones o convergencia más lenta.

SVM: aunque en este caso no presente diferencias en su rendimiento, el modelo también es sensible a la escala porque construye un hiperplano óptimo maximizando márgenes basados en distancias.

Fase III: Reflexión Arquitectónica

En un sistema real de predicción de impago de préstamos, no basta con que el modelo tenga buenas métricas. También es fundamental que todo el proceso esté bien diseñado desde el punto de vista arquitectónico. Esto significa pensar dónde se transforman los datos, cómo se controla el uso de la información, cómo se guardan las versiones del modelo y qué riesgos pueden aparecer si no se hace correctamente.

- **¿Dónde debe ocurrir la transformación de datos en un sistema productivo?**

  La transformación de datos debe ocurrir dentro del mismo flujo del modelo, no antes ni por separado. En un entorno productivo, lo más recomendable es que el preprocesamiento (escalado, codificación de variables categóricas, imputación de valores faltantes, etc.) esté integrado dentro de un pipeline.

  Esto es importante porque:
  - Garantiza que los datos nuevos se transformen exactamente igual que los datos usados en entrenamiento.
  - Evita errores humanos al aplicar transformaciones manuales.
  - Reduce el riesgo de inconsistencias entre entrenamiento y producción.

  Si la transformación se hace fuera del modelo (por ejemplo, en un archivo aparte o manualmente), puede ocurrir que los datos de producción no pasen por el mismo proceso, generando resultados incorrectos. En sistemas financieros, esto puede llevar a aprobar préstamos a personas con alto riesgo o rechazar clientes confiables. Por ende, la transformación debe formar parte del sistema automatizado y no depender de pasos manuales.

- **¿Cómo se versionan modelos y transformadores?**

  Versionar significa guardar un historial de las distintas versiones del modelo y del proceso de transformación, para poder saber cuál se usó en cada momento.

  En un sistema productivo, no solo se versiona el modelo, sino también:
  - El conjunto de datos utilizado.
  - Las transformaciones aplicadas.
  - Los parámetros del modelo.
  - Las métricas obtenidas.

  Esto permite:
  - Reproducir resultados en el futuro.
  - Comparar versiones nuevas contra versiones anteriores.
  - Volver a una versión anterior si la nueva presenta problemas.

  Sin versionamiento, sería imposible rastrear qué modelo tomó determinadas decisiones, lo cual es especialmente delicado en el sector financiero, donde las decisiones deben ser justificables y los procesos son auditados.

- **¿Qué riesgos existen si no se controla el data leakage?**

  El data leakage ocurre cuando el modelo tiene acceso a información que no debería tener al momento de entrenarse. Esto puede hacer que las métricas parezcan muy buenas, pero en realidad el modelo no generaliza bien.

  Los principales riesgos son:
  - Métricas artificialmente altas.
  - Falsa confianza en el modelo.
  - Malas decisiones en producción.
  - Pérdidas económicas.

  En nuestro ejercicio, por ejemplo, escalar los datos antes de dividir en entrenamiento y prueba puede generar fuga de información. Esto hace que el modelo vea datos que en la realidad no debería conocer.
  En un sistema real de préstamos, esto podría significar que el modelo parece excelente en pruebas, pero cuando se usa con clientes reales, su desempeño cae drásticamente, controlar el data leakage es un procedimiento clave en la arquitectura del sistema.

- **¿Qué rol cumple la gobernanza del dato en este proceso?**

  En un sistema de predicción de impago, la gobernanza cumple varios roles:
  - Asegurar la calidad de los datos sin errores, sin duplicados, sin valores inconsistentes.
  - Garantizar que los datos sean confiables y actualizados.
  - Proteger información sensible de los clientes.
  - Documentar cómo se usan los datos en el modelo.

  Además, la gobernanza ayuda a cumplir regulaciones legales, especialmente en el sector financiero, donde las decisiones automatizadas deben poder explicarse.

  Si no existe una buena gobernanza:
  - Se pueden usar datos incorrectos o incompletos.
  - Se pueden violar normas de privacidad.
  - Se pierde trazabilidad sobre cómo se entrenó el modelo.

Fase IV: Conclusiones del proceso y análisis de datos

Conclusiones:

- La data utilizada presenta categorías objetivo (target) desbalanceadas, lo cual suele ser un reto común en la realización de modelos de riesgo de crédito en identificar el default de un cliente.

- Distintos modelos de aprendizaje tienen sus detalles, como considerar los datos que deben ser ingresados o conocer el algoritmo que se desarrolla por detrás a fin de evitar una estimación errónea.

- En la práctica de Python al realizar un correcto tratamiento de la data (no realizar la normalización antes de separar la data) presenta un rendimiento similar al obtenido en Orange Data Mining (cercano a 0.7), lo cual evidencia la importancia de emplear las buenas prácticas.

- **¿Qué aspectos del proceso fueron más aplicables o efectivos?**
  Uno de los aspectos más efectivos fue el uso de pipelines, ya que permitió integrar el preprocesamiento y el modelo en un solo flujo, evitando errores como el data leakage y asegurando que las transformaciones se aplicaran correctamente. Comparar el enfoque incorrecto con el correcto ayudó a entender cómo una mala práctica puede generar resultados engañosos.
  También fue fundamental el análisis exploratorio inicial, porque permitió identificar el desbalance en la variable objetivo solo 11% de incumplimiento y comprender por qué el accuracy podía verse alto sin que el modelo realmente detectara bien el riesgo. Usar ROC AUC en lugar de solo accuracy fue clave para evaluar correctamente la capacidad del modelo para diferenciar entre clientes que pagan y los que incumplen.

- **¿Cuáles fueron los elementos más complejos o desafiantes?**
  Uno de los mayores desafíos fue entender el impacto del data leakage, ya que el modelo incorrecto mostraba un accuracy alto que parecía positivo, pero el ROC AUC reveló que no estaba diferenciando bien entre las clases. Esto permitió comprender la importancia de una arquitectura adecuada.
  También fue complejo manejar el desbalance de clases, porque el modelo tendía a favorecer la clase mayoritaria. Usar class_weight='balanced' ayudó a mejorar la detección de incumplimientos, aunque el accuracy bajara.
  Organizar correctamente todo el proceso separar variables, definir el preprocesamiento y usar validación cruzada requirió cuidado para evitar errores y asegurar que el modelo fuera reproducible y confiable.

Fase V: Aplicación al entorno profesional

Juan: Mi entorno profesional está relacionado directamente con esta práctica al ser de riesgos financieros, el comprender sobre la fuga de datos a fin de evitar sobrestimar los modelos es algo que desconocía y me parece sumamente valioso ya que, por experiencia, he pasado por el ejemplo expuesto de "el modelo dice tener un buen rendimiento, pero no se acerca a la realidad con nuevos datos". También el reforzar conocimientos de prueba del modelo, como validación cruzada, es muy valioso para procesos de optimización en el testeo.

Jonathan: Lo aprendido durante esta semana puede aplicarse directamente en mi trabajo en broker de seguros especialmente en el análisis de riesgo y la evaluación de clientes. Comprender la importancia de una correcta limpieza y transformación de datos, así como evitar el data leakage, permite construir análisis más confiables y tomar decisiones mejor fundamentadas al momento de emitir pólizas o evaluar siniestros.
Además, el uso de métricas adecuadas y el manejo del desbalance de datos es clave en el sector asegurador, donde los casos de fraude o siniestro suelen ser minoritarios. Aplicar estas buenas prácticas ayudaría a mejorar la identificación de clientes de mayor riesgo y fortalecer la toma de decisiones estratégicas dentro de la empresa.